THE NEW STACK

Kubernetes in 2027

Building platforms for AI, autonomy, and scale

Chris J. Preimesberger

sponsored by HPE

The New Stack

Kubernetes in 2027: Building platforms for AI, autonomy, and scale

Ebook Team:

Ben Kubany, Project Manager

Chris J. Preimesberger, Author

Diana Gonçalves, Designer

Supporting Team:

Andrea Washington, Account Director

Andrew Tillery, Senior Marketing Manager

Nick Lucchesi, Editor-in-Chief

Laila Bailey, Email Marketing Manager

Tony Sala, Director of Sales

Brandon Boland, Senior Account Growth Manager

© 2026 *The New Stack*. All rights reserved.

20261022

Table of Contents

Sponsor 4

Introduction 5

Checkpoint #10 7

Checkpoint #9 9

Checkpoint #8 11

Checkpoint #7 13

Checkpoint #6 15

Checkpoint #5 17

Checkpoint #4 19

Checkpoint #3 21

Checkpoint #2 23

Checkpoint #1 25

What's next 27

The 2027 Kubernetes checklist 28

About the author 29

Disclosure 30

Sponsor

We are grateful for the support of our sponsor for this ebook:

HPE combines AI, data, and deep expertise to deliver transformation. We use the power of AI, cloud, and networking to help customers move faster and work smarter.

INTRODUCTION

Enterprise IT is adapting to AI workloads, greater automation, and expanding operational scale.

Kubernetes, originally developed at Google and open-sourced in 2014, provides an open-source platform for deploying, scaling, and managing containerized applications across clusters. It provides key building blocks for an enterprise platform; application reliability, security, and cost control also depend on how teams configure and operate it.

Preparing for 2027 means translating these capabilities into clear operational priorities. Teams need to define service ownership, user expectations, and which changes can be safely automated. Those decisions should guide architecture, governance, and capacity planning as AI inference and agent workloads grow.

Platform engineering increasingly includes AI applications, variable resource demands, and recovery workflows across hybrid and multicloud environments. Poorly defined workflows can create operational friction, excess infrastructure costs, and security risks. Planning should establish access policies, shared governance baselines, and measurable service targets before workloads enter production.

The industry experts interviewed for this ebook emphasize designing processes before automating them, measuring systems before launch, and tying AI services to business outcomes. Use these readiness checks as planning recommendations: set thresholds, review cadences, and approval requirements for each workload, and validate them with evidence.

Looking ahead to 2027, here are 10 priorities for chief technology officers aiming to deploy Kubernetes to its fullest, most effective potential. The checkpoints count down from 10 to 1 for reading convenience; the numbering does not indicate priority or an implementation sequence.

Checkpoint #10

Perform pre-launch stress and resilience drills under AI traffic. Run realistic load simulations, failure injections, and demand-spike stress tests on model-serving clusters before production deployment.

Why it matters:

Tests reveal how the service behaves during the load and failure scenarios exercised. They provide evidence for capacity and recovery decisions and help identify gaps before launch.

Model-serving systems need tests that reflect the intended workload. Before running these drills, agree on acceptable latency, throughput, cost, and recovery targets for the workload. Prompt length, output length, concurrency, GPU memory, cache behavior, and model-loading time can affect capacity and startup delays. Traffic spikes can fill queues and increase latency. Include dependencies such as model storage,

databases, networking, identity services, and external model endpoints in the test plan.

Exercise the components actually used in the serving stack. For example, vLLM and Nvidia Triton Inference Server serve models; KEDA supports event-driven workload autoscaling, while Kueue manages workload admission and quotas. Test batching, queueing, pod scaling, and node provisioning end to end. Record where capacity is exhausted and how long additional accelerator capacity takes to become usable.

In a controlled environment, simulate node or model-service outages, storage delays, and network failures. Check whether configured retries, fallback routes, admission limits, and graceful degradation behave as intended. Define the acceptable service impact and data-loss tolerance before the test, measure actual recovery, and assign owners to unresolved gaps.

Suggested readiness checks:

- **Measured scaling behavior:** Record queueing, pod and node startup times, available accelerator capacity, and latency under spikes. Verify behavior when additional capacity is unavailable.
- **Tested failure recovery:** Measure recovery time, failed or retried requests, and any data loss in each exercised scenario. Obtain sign-off on remaining risks before production.

Checkpoint #9

Monitor AI output quality alongside infrastructure metrics.

Define evaluation datasets and scoring criteria for task success, factual correctness, unsupported answers, and safety, and assess them alongside service performance.

Why it matters:

A model endpoint can be available and fast while returning poor answers. Quality checks should inform release and incident decisions, with response actions agreed in advance.

Infrastructure availability is only one part of application health, for conventional services and AI. A large language model endpoint can respond quickly while producing incorrect or irrelevant output. Evaluate the service against the task users need to complete, including the quality of retrieved context and the consequences of a wrong answer.

Model evaluation requires expertise beyond infrastructure monitoring. Data science and domain teams should define representative tests and acceptance criteria, while platform engineers monitor capacity and performance; business and security owners contribute their requirements. Re-evaluate relevant quality and performance measures after changes to the model, prompt, retrieval, or serving configuration. Automated scores are useful evidence but need validation and, where appropriate, human review.

Canary and blue-green deployment strategies can incorporate quality checks alongside availability and latency metrics. Configure the release system to pause, reject, or reverse a rollout when validated criteria are breached. Test the previous model and configuration as a recovery option, and define how teams handle inconclusive evaluations. These behaviors depend on the delivery pipeline and its policies.

Suggested readiness checks:

- **Representative quality evaluation:** Use controlled test sets and appropriately protected production samples. Document scoring methods, thresholds, failure examples, and the limits of automated evaluation.
- **Agreed release decisions:** Assign owners for quality regressions and test pause, traffic-shift, and rollback procedures. Correlate quality changes with model, prompt, retrieval, and infrastructure changes before selecting a remedy.

Checkpoint #8

Standardize the operating process **across** clusters without standardizing every runtime. Apply shared security, access, and networking baselines across public, private, and edge clusters while allowing environment-specific configurations. Keep Kubernetes APIs and the tools teams already use; standardize the request, governance, automation, and lifecycle processes around them.

Why it matters:

Shared baselines help reduce configuration drift and inconsistent access controls as environments expand. Teams still need to define exceptions, detect violations, and decide which remediation actions to automate.

As Kubernetes environments expand across public clouds, data centers, and edge deployments, managing clusters independently increases operational overhead. Inconsistent settings across teams lead to security risks: ingress rules diverge, role-based access control (RBAC) roles grant broader access in secondary environments, and network policies fall behind corporate security guidelines.

Use version-controlled configuration and GitOps workflows to distribute approved baselines. Policy engines such as OPA Gatekeeper and Kyverno can evaluate Kubernetes resources against configured rules. Admission checks can reject noncompliant requests, while audits can identify existing violations. Mutation and remediation capabilities differ by tool and policy type; configure and test them explicitly.

Preserve environment-specific requirements within the shared baseline. Managed cloud services, data residency constraints, and intermittently connected edge clusters can require different settings. Assign owners and expiration or review conditions to exceptions, and record when each cluster last reported its state.

Suggested readiness checks:

- **Tested policy enforcement:** Verify admission, audit, and any remediation rules separately. Track violations, approved exceptions, and the time required to resolve drift.
- **Fleet visibility:** Show policy status, missing telemetry, and how old each cluster's last report is. Use this evidence in compliance reviews; a dashboard alone cannot establish regulatory compliance.

Checkpoint #7

Audit and instrument AI agent tool execution: Implement logging and observability tracking an agent's tool usage, permissions, task outcomes, and resource consumption.

Why it matters:

Infrastructure events alone do not establish an agent's task, authority, or outcome. Correlate tool activity with the initiating task and approved permissions while protecting sensitive inputs and outputs.

When auditing is enabled, Kubernetes audit logs record API activity according to the configured audit policy; Fluentd collects logs, and Loki stores and queries aggregated logs. Agent-level instrumentation supplies additional task and tool context. Model Context Protocol (MCP) is an open standard for connecting AI applications to external systems, including tools and data sources. If an MCP server exposes

authorized infrastructure tools, a task may involve reading logs, changing scaling settings, or invoking cloud APIs. The protocol itself does not grant permission to perform those actions. Audit records should show whether an agent acted within its approved permissions and operating limits; checkpoint 6 addresses how to establish those boundaries.

Capture task or session identifiers, tool names, timestamps, authorization decisions, errors, and outcomes at the agent integration layer. Record necessary inputs and outputs only under an explicit data-handling policy. Prompts, retrieved documents, and tool results may contain secrets or personal information: minimize or redact content, restrict access, and define retention. Tool traces show observable behavior, not a reliable record of a model's internal reasoning.

Agent telemetry can also support financial governance. Correlate available token counts, model API charges, and infrastructure usage with workflows. Define how shared costs are allocated and how retries or failed tasks are counted. Cost estimates and alerts need a separate enforcement mechanism if spending should block or stop a task.

Suggested readiness checks:

- **Contextual audit trails:** Link changes and tool calls to a task identifier, agent identity, permission decision, and outcome. Test redaction, access controls, and retention for any captured content.
- **Agent cost accountability:** Track available usage and attributed costs per workflow or session. Verify that configured spending limits, alerts, and stop controls operate as intended.

Checkpoint #6

Enforce guardrails for AI infrastructure agents. Define identities, least-privilege permissions, operational and spending limits, and approval requirements before allowing agents to modify production clusters.

Why it matters:

Agent-generated actions can be nondeterministic and may have unintended effects. Start with read-only diagnostics or confined testing and expand authority only after validating permitted actions and recovery procedures.

As AI agents move from monitoring tools to active management systems, unchecked automated actions introduce operational risks. An agent making automated cluster changes could trigger service disruptions, such as unnecessarily reallocating accelerator nodes, modifying ingress rules, or stopping critical pods while resolving localized issues.

Give each agent an identifiable principal and a human owner. Apply least-privilege RBAC and restrict sensitive operations, including access to secrets, workload execution, and changes to core system resources. The kube-system namespace contains Kubernetes system objects. RBAC governs API permissions; admission policies, workload isolation, and controls on the agent's tools also limit what permitted actions can do.

Set resource limits, maximum scaling bounds, and separate monetary budget controls. Require human approval for higher-risk changes, with policy checks before execution. Admission controllers and policy engines such as Open Policy Agent can evaluate requests, but you must explicitly implement and connect the approval workflow. Keep permitted actions narrow, auditable, and recoverable.

Suggested readiness checks:

- **Confined execution scopes:** Begin with read-only production diagnostics or isolated non-production tasks. Authorize routine changes incrementally, with named owners and tested limits.
- **Verified change gates:** Test that the system rejects unauthorized changes and enforces required approvals. Retain human review for higher-risk actions and establish a way to halt automation and recover.

Checkpoint #5

Define performance targets for your AI inference workload.

Establish pre-launch baselines that separate time to first token (TTFT) from end-to-end response latency, along with throughput, GPU utilization, and cost per request.

Why it matters:

High GPU utilization does not guarantee a fast or cost-effective user experience. Profile representative workloads before launch so teams have a measured baseline for latency, throughput, capacity, and cost.

"Profile it before you launch," says Mahesh Ramachandran, vice president of product management for HPE OpsRamp Software. The platform provides monitoring and observability across hybrid IT environments. CPU usage, request counts, pod health, and GPU utilization remain useful, but they do not fully describe inference service performance. Queueing, prompt and output lengths, batching,

memory constraints, model routing, and dependencies can affect response times. Investigate these signals together when resource utilization looks normal, but users report slow responses.

Measure TTFT as the interval from a request to the first output token at a defined observation point, and measure end-to-end latency through completion. Inter-token latency describes gaps between streamed outputs; time per output token summarizes generation speed per request. Document the serving stack's definitions and whether measurements include network and queueing time. Set targets for the intended interaction and compare latency distributions across representative prompt lengths, output lengths, and concurrency levels.

Use those measurements to choose and test scaling signals, such as queue depth or concurrent requests. KEDA can drive workload autoscaling through Kubernetes' Horizontal Pod Autoscaler. Kueue manages workload admission and resource quotas; vLLM is an inference and serving engine. These components serve different roles. Evaluate cost per request or completed task alongside service targets and enforce budgets and capacity limits through separately configured controls.

Suggested readiness checks:

- **Measured latency baselines:** Test TTFT, generation speed, end-to-end latency, errors, and throughput under representative sustained load and spikes. Record the model, hardware, workload mix, and target service-level objectives (SLOs).
- **Cost-aware capacity:** Validate scaling triggers, maximum capacity, and budget controls together. Include accelerator availability and model-loading time in the capacity plan.

Checkpoint #4

Observe Kubernetes in context, not in isolation. Start with Kubernetes-native health and telemetry, then connect those signals to applications, VMs, cloud services, networks, and infrastructure beyond the cluster using tools such as HPE OpsRamp Software where configured.

Why it matters:

Healthy node and pod metrics can coexist with slow user requests. Correlating application and infrastructure signals helps teams investigate the complete service, including dependencies outside their administrative control.

Standard cluster metrics can present an incomplete view of overall system health. Dashboards may report functional pod replicas, sufficient CPU capacity, and normal node states while users experience slow response times. Microservices frequently rely on external databases, on-premises systems, gateways, and third-party APIs. When performance drops outside the cluster, isolated container metrics

can show healthy operation even when users are impacted.

HPE OpsRamp Software is a SaaS-based IT operations management (ITOM) platform for monitoring, observability, event and incident management, and automation across hybrid IT environments. Where configured, its discovery and monitoring capabilities can relate Kubernetes signals to physical, virtual, cloud, network, and application context aligned to service health. Coverage depends on the agents, integrations, permissions, and telemetry configured for the environment. Instrument critical paths first; where a third-party service cannot expose internal traces, measure its response times and availability through boundary measurements or synthetic transactions. Treat correlation as evidence for investigation, not as guaranteed root cause.

Suggested readiness checks:

- **Cross-boundary evidence:** Trace critical request paths where instrumentation is available. Use synthetic checks and dependency response measurements to cover external services, and document remaining blind spots.
- **Correlated diagnosis:** Align application latency, errors, and throughput with relevant logs, events, traces, and infrastructure metrics. Validate suspected causes against user experience rather than treating correlation alone as proof.

Checkpoint #3

Implement automated upgrade readiness and recovery checks. Run automated pre-upgrade checks and regular restoration drills before installing version updates.

Why it matters:

Pre-upgrade checks and restoration drills reduce maintenance risk and expose recovery gaps. Set the testing cadence according to workload criticality, change frequency, and recovery objectives, and repeat relevant checks after material changes.

Keep Kubernetes within supported versions and plan upgrades against the target release's requirements. Removed APIs can break clients or manifests; custom resources, controllers, networking, and storage add-ons can have their own compatibility constraints. Test the actual application stack and follow the distribution or service provider's supported upgrade path.

Configure pre-flight checks for deprecated or removed APIs, component compatibility, available capacity, and pod disruption budgets (PDBs). PDBs constrain supported voluntary evictions; they do not prevent every outage or govern all rollout behavior. Define which failed checks stop the pipeline and require remediation before an upgrade proceeds.

Recovery drills should test both cluster configuration and application data. An etcd snapshot preserves Kubernetes state, while persistent volumes and external databases need their own protection and restoration procedures. After recovery, validate credentials, storage access, and application consistency. Verify the platform and version's supported recovery path instead of assuming you can always reverse an upgrade.

Suggested readiness checks:

- **Automated pre-flight audits:** Run compatibility and policy checks before maintenance and include suitable checks in CI/CD. Record failures, exceptions, and remediation owners.
- **Measured recoverability:** Test restoration against workload-specific recovery time objectives (RTOs)—the targeted time to restore a business process—and recovery point objectives (RPOs)—the maximum acceptable data loss measured in time. Record actual results and unresolved gaps.

Checkpoint #2

Define a post-deployment operational ownership matrix. Map a task-responsibility matrix that divides duties among developers, platform engineers, and operations teams for configuration changes, updates, capacity planning, and incident response.

Why it matters:

Documented responsibilities clarify who acts, who approves, and who is contacted when a production service needs attention. Agree on escalation paths and service objectives before an incident.

Deploying an application to production is one step in its operational lifecycle. Across multiple clusters, unclear ownership can delay diagnosis and recovery. The ownership matrix should answer practical questions before they become urgent:

- *Does a node failure fall to the platform team? Did an unconstrained service consume available memory?*

- *Who owns ingress certificate renewal, upstream API deprecation, or snapshot restores?*

HPE Morpheus Software helps platform teams manage self-service provisioning, governance, and lifecycle operations. Karthik Subramanian, principal product manager for HPE Morpheus Software, describes a practical division of labor: platform teams maintain the Kubernetes environment. In contrast, application teams build and deploy the workloads that use it. A RACI matrix identifies who is responsible, accountable, consulted, and informed. Adapt those assignments to the operating model, and agree explicitly on shared areas such as autoscaling, capacity, access, backups, and recovery; managed-service providers may own parts of the infrastructure.

Test these boundaries in incident exercises. Record who can authorize a change, who executes it, and how the next team receives the evidence it needs. Measure time to identify and resolve problems, and revise handoffs that slow recovery.

Suggested readiness checks:

- **Explicit lifecycle sign-offs:** Assign owners and escalation paths for upgrades, configuration drift, capacity changes, access, and recovery. Document expected service impact and approval requirements.

- **Defined service targets:** Set service-level objectives (SLOs) for measurable indicators such as availability and latency. Distinguish those operational targets from service-level agreements (SLAs), which specify commitments and any consequences for missing them.

Checkpoint #1

Install guardrails for your developer self-service. Build service catalog templates that connect approved resource choices with access policies, cost controls, and monitoring requirements. HPE Morpheus Software can support governed self-service across Kubernetes, VM, and cloud services while preserving direct access to Kubernetes tools.

Why it matters:

A curated catalog can reduce provisioning delays while helping teams manage resource use and policy risk. Routine, low-risk requests can follow pre-approved workflows; exceptions and higher-risk changes still need review.

"Automation doesn't fix the process," says Marius Bogoevici, senior principal product manager at HPE. Effective developer access balances speed with operational boundaries. Central ticket queues and repeated manual handoffs can delay delivery. Unrestricted developer access to infrastructure can create another problem: misconfigured resources, unclear ownership, and hard-to-allocate costs.

HPE Morpheus Software helps platform teams standardize how Kubernetes and VM services are requested, provisioned, governed, automated, and managed through their lifecycle. HPE Morpheus Enterprise Software can manage HPE Morpheus Kubernetes Service (HKS), supported existing Kubernetes environments, VMware vCenter/ESXi, HPE's HVM hypervisor, and public-cloud infrastructure. Developers retain direct Kubernetes access through familiar APIs and tools, while platform teams apply blueprints, workflows, role-based access control (RBAC), approvals, placement policies, and other guardrails. The aim is Kubernetes without another operational silo: one fulfillment process can serve Kubernetes and VM-based services without forcing every workload onto the same runtime.

Kubernetes resource quotas limit aggregate resource consumption within a namespace, such as CPU, memory, storage, and object counts. Monetary controls require separate pricing, budget policies, and cost monitoring.

Assign each provisioned environment an owner and a review or expiration date, and configure audit records so teams can identify who requested and changed resources.

Suggested readiness checks:

- **Bounded self-service:** Measure provisioning time, successful requests, exceptions, and actual resource use for the approved catalog. Keep a clear route for requests outside it.
- **Tested governance:** Verify that configured access, resource, and budget policies work as intended. Regularly review exceptions, idle environments, and unexpected spending.

What's next

Readiness for 2027 should connect technical evidence to a business decision. Agree on the user or business outcome the service must improve, then have platform, application or data science, security, and business owners review the relevant capacity, quality, cost, and recovery results. Maintain the ten checks as workloads and operating conditions change. The aim is Kubernetes without another operational silo: a platform whose capabilities and limits are measured, whose responsibilities are explicit, and whose automation operates within tested boundaries—while Kubernetes, VMs, and cloud services retain the operating models and technologies that fit each workload.

HPE

SUMMARY

HPE Morpheus Enterprise Software is HPE's hybrid cloud management platform for self-service provisioning, orchestration, and policy-based governance across environments. Platform teams can configure blueprints, role-based access control (RBAC), and policy-based governance, including budget guardrails.

KEY FEATURES

- unifies cloud operations
- provides self-service provisioning
- provides application lifecycle governance across virtual machines (VMs), containers, bare metal, and public/private clouds

Learn more at HPE.com.

The 2027 Kubernetes checklist

10	◯	Stress-test inference workloads and rehearse failure recovery before launch.
09	◯	Evaluate AI output quality alongside service performance.
08	◯	Apply shared governance across clusters, with documented local exceptions.
07	◯	Audit agent tool use, outcomes, and costs while protecting sensitive data.
06	◯	Set permissions, spending limits, and approval gates for AI agents.
05	◯	Benchmark AI inference latency, throughput, capacity, and cost.
04	◯	Connect application and infrastructure observability across hybrid dependencies.
03	◯	Test upgrade readiness and recovery against workload-specific objectives.
02	◯	Assign ownership for Kubernetes operations, changes, and incident response.
01	◯	Establish developer self-service with approved templates, access policies, and cost controls.

About the author



Chris J. Preimesberger is a contributing writer for *The New Stack* who has covered IT since the dawn of the internet. He is proficient in sectors that include IT innovation, storage, networking, cybersecurity, development trends, and AI-enabled systems.

He previously served as editor-in-chief of eWEEK (2011 to 2021), where he hosted eSPEAKS video interviews and wrote more than 5,000 articles during his tenure. He won a Folio Award in 2011 for his eWEEK cover story on Salesforce and its founder, Marc Benioff. He also reported in 2006 on cybersecurity red teams at Sandia National Laboratories in Albuquerque, New Mexico.

In his long career, his work has been published in the *Chicago Tribune, the Los Angeles Daily News, San Jose Mercury News, San Francisco Examiner, Orlando Sentinel, Peninsula Times Tribune (Palo Alto, California), Dr. Dobbs' Journal*, and *SD Times*, and on tech sites such as *eWEEK, DevX, TechWeekEurope, PC Magazine, Baseline, CNET.com*, and *TechRepublic*.

Disclosure

The following companies mentioned in this ebook are sponsors of *The New Stack*: HPE, Google, and Nvidia.